

Chương 1: FEATURE ENGINEERING

Áp dụng các kỹ thuật để chuẩn hoá vector đặc trưng.

I. Các bước thực hiện chuẩn hoá vector đặc trưng

1. Đọc dữ liệu

Thực hiện đọc dữ liệu từ file CSV vào List.

2. Rescaling

- Tìm giá trị min và max của từng cột
- Sử dụng giá trị min và max tìm được để áp dụng chuẩn hoá dữ liệu

3. Standardization

- Tìm giá trị Mean (giá trị trung bình) và Standard deviation (độ lệch chuẩn) của từng cột trong tập dữ liệu.
- Sử dụng giá trị mean và std tìm được để áp dụng chuẩn hoá.

4. Thực hiện One-hot encoding với dữ liệu rời rạc

Đối với các dữ liệu categorical là các dữ liệu rời rạc, chúng ta cần phải chuyển dữ liệu về dưới dạng one-hot encoding.

II. Thực hiện

1. Đọc dữ liệu

Dữ liệu được đọc vào bởi thư viện `csv`, sử dụng hàm `reader` để tiến hành đọc dữ liệu. Dữ liệu đọc được sẽ được đưa vào list, mỗi điểm dữ liệu (data point) là một hàng (row) và mỗi feature là một cột (column)

```
In [21]: #Đọc dữ liệu
import csv

with open('/content/pima-indians-diabetes.csv', newline='') as f:
    reader = csv.reader(f)
    data = list(reader)

print(data)
```

6', '35.1', '0.692', '30', '1'], ['10', '179', '70', '0', '0', '35.1', '0.200', '37', '0'], ['2', '102', '86', '36', '120', '45.5', '0.127', '23', '1'], ['6', '105', '70', '32', '68', '30.8', '0.122', '37', '0'], ['8', '118', '72', '19', '0', '23.1', '1.476', '46', '0'], ['2', '87', '58', '16', '52', '32.7', '0.166', '25', '0'], ['1', '180', '0', '0', '0', '43.3', '0.282', '41', '1'], ['12', '106', '80', '0', '0', '23.6', '0.137', '44', '0'], ['1', '95', '60', '18', '58', '23.9', '0.260', '22', '0'], ['0', '165', '76', '43', '255', '47.9', '0.259', '26', '0'], ['0', '117', '0', '0', '0', '33.8', '0.932', '44', '0'], ['5', '115', '76', '0', '0', '31.2', '0.343', '44', '1'], ['9', '152', '78', '34', '171', '34.2', '0.893', '33', '1'], ['7', '178', '84', '0', '0', '39.9', '0.331', '41', '1'], ['1', '130', '70', '13', '105', '25.9', '0.472', '22', '0'], ['1', '95', '74', '21', '73', '25.9', '0.673', '36', '0'], ['1', '0', '68', '35', '0', '32.0', '0.389', '22', '0'], ['5', '122', '86', '0', '0', '34.7', '0.290', '33', '0'], ['8', '95', '72', '0', '0', '36.8', '0.485', '57', '0'], ['8', '126', '88', '36', '108', '38.5', '0.349', '49', '0'], ['1', '139', '46', '19', '83', '28.7', '0.654', '22', '0'], ['3', '116', '0', '0', '0', '23.5', '0.187', '23', '0'], ['3', '99', '62', '19', '74', '21.8', '0.279', '26', '0'], ['5', '0', '80', '32', '0', '41.0', '0.346', '37', '1'], ['4', '92', '80', '0', '0', '42.2', '0.237', '29', '0'], ['4', '137', '84', '0', '0', '31.2', '0.252',

Dữ liệu đọc được từ file CSV là dữ liệu có kiểu `text` . Cần phải chuyển đổi kiểu dữ liệu từ `text` sang `float` . Sử dụng 2 vòng lặp lồng nhau để tiến hành.

```
In [25]: dataset = data
for row in range(len(dataset)):
    for col in range(len(dataset[row])):
        dataset[row][col] = float(dataset[row][col])
```

2. Tìm giá trị Min - Max

Tại bước này cần tìm giá trị min và max của từng cột (feature). Giá trị tìm được là một list, trong đó mỗi phần tử của list chứa giá trị min và max của mỗi cột.

Trong đoạn code dưới đây sử dụng hàm `min` và `max` để tìm giá trị lớn nhất và nhỏ nhất.

```
In [27]: def dataset_minmax(dataset):
    minmax = list()
    for i in range(len(dataset[0])):
        col_values = [row[i] for row in dataset]
        value_min = min(col_values)
        value_max = max(col_values)
        minmax.append([value_min, value_max])
    return minmax
```

```
In [29]: minmax = dataset_minmax(data)
```

```
In [30]: minmax
```

```
Out[30]: [[0.0, 17.0],
          [0.0, 199.0],
          [0.0, 122.0],
          [0.0, 99.0],
          [0.0, 846.0],
          [0.0, 67.1],
          [0.078, 2.42],
          [21.0, 81.0],
          [0.0, 1.0]]
```

```
In [31]: # Rescale dataset columns to the range 0-1
def normalize_dataset(dataset, minmax):
    for row in dataset:
        for i in range(len(row)):
            row[i] = (row[i] - minmax[i][0]) / (minmax[i][1] - minmax[i][0])
```

```
In [33]: normalize_dataset(data, minmax)
```

3. Chuẩn hoá theo phương pháp rescaling.

Để tiến hành rescale cần phải tìm được giá trị kỳ vọng (mean) và độ lệch chuẩn (std). Mean được tính bằng tổng giá trị của cột chia cho số điểm dữ liệu. Std được tính bằng cách áp dụng công thức tính độ lệch chuẩn:

$$std = \sqrt{\frac{\sum(x - \bar{x})^2}{n}}$$

```
In [38]: from math import sqrt
```

Tìm giá trị trung bình

```
In [35]: # calculate column means
def column_means(dataset):
    means = [0 for i in range(len(dataset[0]))]
    for i in range(len(dataset[0])):
        col_values = [row[i] for row in dataset]
        means[i] = sum(col_values) / float(len(dataset))
    return means
```

Tìm độ lệch chuẩn

```
In [36]: # calculate column standard deviations
def column_stdevs(dataset, means):
    stdevs = [0 for i in range(len(dataset[0]))]
    for i in range(len(dataset[0])):
        variance = [pow(row[i]-means[i], 2) for row in dataset]
        stdevs[i] = sum(variance)
    stdevs = [sqrt(x/(float(len(dataset)-1))) for x in stdevs]
    return stdevs
```

```
In [39]: # Estimate mean and standard deviation
means = column_means(dataset)
stdevs = column_stdevs(dataset, means)
```

Áp dụng chuẩn hoá

```
In [41]: # standardize dataset
def standardize_dataset(dataset, means, stdevs):
    for row in dataset:
        for i in range(len(row)):
            row[i] = (row[i] - means[i]) / stdevs[i]
```

```
In [42]: standardize_dataset(dataset, means, stdevs)
```

III. Sử dụng thư viện

Trong nội dung này các bạn có thể thực hiện chuẩn hoá bằng cách sử dụng thư viện sklearn để chuẩn hoá.

```
In [48]: from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

```
In [46]: scaler = MinMaxScaler()
# transform data
scaled = scaler.fit_transform(dataset)
print(scaled)
```

```
[[1.00000000e+00 1.00000000e+00 1.00000000e+00 ... 6.27000000e-01
 1.00000000e+00 1.00000000e+00]
 [9.80392157e-03 2.88605188e-03 7.51366120e-03 ... 1.16567037e-01
 3.33333333e-03 0.00000000e+00]
 [7.84313725e-02 6.21349993e-03 7.28597450e-03 ... 2.53629377e-01
 3.66666667e-03 1.00000000e+00]
 ...
 [4.90196078e-02 4.10837974e-03 8.19672131e-03 ... 7.13065756e-02
 3.00000000e-03 0.00000000e+00]
 [9.80392157e-03 4.27814749e-03 6.83060109e-03 ... 1.15713066e-01
 8.66666667e-03 1.00000000e+00]
 [9.80392157e-03 3.15768029e-03 7.96903461e-03 ... 1.01195559e-01
 6.66666667e-04 0.00000000e+00]]
```

```
In [49]: scaler = StandardScaler()
# transform data
scaled = scaler.fit_transform(dataset)
print(scaled)

[[ 2.00678965e+01  2.76821488e+01  2.76424551e+01 ...  3.22014963e+00
  2.75318856e+01  1.36589591e+00]
 [-6.08004994e-01 -6.99787765e-02 -4.58906820e-02 ... -3.66228188e-01
 -5.63702343e-02 -7.32120209e-01]
 [ 8.24978280e-01  2.26322666e-02 -5.22426769e-02 ...  5.96792171e-01
 -4.71433929e-02  1.36589591e+00]
 ...
 [ 2.10842591e-01 -3.59583933e-02 -2.68346972e-02 ... -6.84234910e-01
 -6.55970757e-02 -7.32120209e-01]
 [-6.08004994e-01 -3.12333401e-02 -6.49466667e-02 ... -3.72228314e-01
  9.12592284e-02  1.36589591e+00]
 [-6.08004994e-01 -6.24186913e-02 -3.31866922e-02 ... -4.74230471e-01
 -1.30184966e-01 -7.32120209e-01]]
```

III. Chuẩn hoá dữ liệu rời rạc

Trong nội dung này chúng ta sẽ thực hành one-hot encoding và label encoding để chuẩn hoá dữ liệu rời rạc

1. Label encoding

Label Encoding là việc chuyển dữ liệu rời rạc về giá trị liên tục. Trong đó mỗi giá trị tương ứng với nhãn ban đầu.

```
In [50]: from numpy import array
from numpy import argmax
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
# define example
data = ['cold', 'cold', 'warm', 'cold', 'hot', 'hot', 'warm', 'cold', 'warm', 'hot']
values = array(data)
print(values)
# integer encode
label_encoder = LabelEncoder()
integer_encoded = label_encoder.fit_transform(values)
print(integer_encoded)

['cold' 'cold' 'warm' 'cold' 'hot' 'hot' 'warm' 'cold' 'warm' 'hot']
[0 0 2 0 1 1 2 0 2 1]
```

Các bạn thấy trong bước này, cold được chuẩn hoá về thành giá trị 0 , warn được chuẩn hoá về thành giá trị 2 , hot được chuẩn hoá về giá trị 1.

One-hot encoding

```
In [53]: # binary encode
onehot_encoder = OneHotEncoder(sparse=False)
integer_encoded = integer_encoded.reshape(len(integer_encoded), 1)
onehot_encoded = onehot_encoder.fit_transform(integer_encoded)
print(onehot_encoded)
```

```
[[1. 0. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]]
```

Sau bước này, `cold` được chuẩn hoá về thành giá trị `[1. 0. 0.]`, `warm` được chuẩn hoá về thành giá trị `[0. 0. 1.]`, `hot` được chuẩn hoá về giá trị `[0. 1. 0.]`.

Bài thực hành

Áp dụng các kỹ thuật trên để chuẩn hoá dữ liệu cho tập dữ liệu:

<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv>
(<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv>)

Bản mô tả dữ liệu:

<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.names> (<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.names>)

In []: